

```

File Edit View Search Terminal Tabs Help
server client
$ ./server
Server waiting

```

Figure 1: Server running

```

File Edit View Search Terminal Tabs Help
server client
$ ./client
Reply from Server: s
$

```

Figure 2: Client's output

```

#include <sys/socket.h>
#include <netinet/in.h>
int main()
{
    int sfd, cfd;
    int ch='k';
    struct sockaddr_in saddr, caddr;

    sfd= socket(AF_INET, SOCK_STREAM, 0);
    saddr.sin_family=AF_INET;      /* Set Address
Family to Internet */
    saddr.sin_addr.s_addr=htonl(INADDR_ANY); /* Any
Internet address */
    saddr.sin_port=htons(29008);   /* Set server port
to 29008 */

    /* select any arbitrary Port >1024
*/

    bind(sfd, (struct sockaddr *)&saddr, sizeof(saddr));
    listen(sfd, 1);
    while(1) {
        printf("Server waiting\n");
        cfd=accept(sfd, (struct sockaddr *)&NULL, NULL);
        if(read(cfd, &ch, 1)<0) perror("read");
        ch++;
        if(write(cfd, &ch, 1)<0) perror("write");
        close(cfd);
    }
}

```

About the code

The first thing to look at is *struct sockaddr_in*. This structure is used to hold an Internet (IP) address in the *sin_addr* member, which is of the type *struct in_addr* and holds a 32-bit unsigned integer. The port number is held in the *sin_port* member, an unsigned 16-bit integer (so port numbers must

be less than 65536).

Next, let's look at a call to *socket()* (the *includes* required are *sys/types.h* and *sys/socket.h*):

```
int socket(int domain, int type, int protocol);
```

A successful call to *socket()* returns a descriptor that will be used for end-point communication. The first argument, *domain*, specifies a communication domain—the protocol family to be used for communication. According to *sys/sockets.h*, these are:

Name	Purpose
AF_UNIX, AF_LOCAL	Local communication
AF_INET	IPv4 Internet protocols
AF_INET6	IPv6 Internet protocols
AF_IPX	IPX—Novell protocols
AF_NETLINK	Kernel user interface device
AF_X25	ITU-T X.25 / ISO-8208 protocol
AF_AX25	Amateur radio AX.25 protocol
AF_ATMPVC	Access to raw ATM PVCs
AF_APPLETALK	AppleTalk
AF_PACKET	Low-level packet interface

AF stands for Address Family. We're using AF_INET here—Internet IPv4. The next argument, *type*, specifies the type of communication, from these choices:

SOCK_STREAM	Sequenced, reliable, two-way, connection-based byte stream (TCP, SCTP etc.)
SOCK_DGRAM	Datagrams—connectionless, unreliable (UDP)
SOCK_SEQPACKET	Sequenced, reliable, two-way connection-based data transmission path for datagrams of fixed maximum length (SCTP)
SOCK_RAW	Raw network protocol access (Transport layer protocols not required)

The *protocol* argument specifies a protocol to be used with the socket. Normally, only a single protocol exists to support a particular socket type within a given family (as specified within parentheses above). In such cases, this argument is 0.

Next, let us set the address in *sin_addr*, as explained above. With the call to *socket()*, a socket is created, but no address is assigned to it. So we have the *bind()* function:

```
int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```